Java Backend Architecture

Interview Series

Chapter 19.9

Observability & SRE Practices

50+ Interview Q&A	Code Examples	Architecture Diagrams	Advanced Topics

Chapter 19.9 – Observability & SRE Practices

Overview

Observability enables understanding the internal state of a Java system from its external outputs: metrics, logs, and traces (the three pillars). Prometheus + Grafana is the standard metrics stack; OpenTelemetry provides vendor-neutral distributed tracing. SRE practices (SLI, SLO, error budgets) provide a framework for reliability engineering and data-driven decision making on Java microservice deployments.

Key Definitions

Term	Definition
Prometheus	Pull-based metrics system. Scrapes /metrics endpoints. PromQL for querying. Alertmanager for alerts.
Micrometer	Java metrics facade. Auto-exposes JVM + Spring Boot metrics to Prometheus (via /actuator/prometheus).
PromQL	Prometheus query language. rate(), histogram_quantile(), increase() for computing latency/error rates.
Alertmanager	Routes Prometheus alerts to PagerDuty/Slack/email. Groups, inhibits, and silences alerts.
Grafana	Visualization platform. Dashboards for Prometheus/Loki/Jaeger/Elasticsearch data sources.
OpenTelemetry (OTel)	Vendor-neutral observability SDK. Java agent auto-instruments Spring Boot for traces + metrics.
Jaeger	Distributed tracing backend. Visualises request flows across Java microservices.
SLI	Service Level Indicator: measurable metric (p99 latency, error rate, availability).
SLO	Service Level Objective: target for an SLI (e.g. 99.9% requests < 200ms p99).
Error Budget	Time/requests allowed to fail while meeting SLO. Consumed by outages; guides reliability investment.

Diagram 1: Observability Stack for Java Microservices

```
Java Spring Boot App
|-- Micrometer (auto) ---> /actuator/prometheus
|-- OTel Agent (auto) ---> traces (OTLP)
|-- Logback (structured) -> stdout (JSON)

Prometheus                    Grafana
+-----------------+          +-----------------+
| Scrape /actuator |  ------> | Dashboards      |
| prometheus every |          | - JVM metrics   |
| 15s              |          | - Spring Boot   |
| PromQL queries   |          | - Business KPIs |
| Recording rules  |          +-----------------+
+-----------------+
       |
Alertmanager --> PagerDuty (critical)
             --> Slack (warning)

Jaeger/Tempo <-- OTel Collector <-- Java OTel Agent
```

Diagram 2: SLO Error Budget Calculation

```
SLO: 99.9% of requests respond in < 200ms over 30 days

Total requests per month: 10,000,000
Allowed failures (0.1%):  10,000 requests

Error Budget Status:
Week 1: 2,000 failures (20% budget consumed) -- on track
Week 2: 5,000 failures (50% budget consumed) -- warning
Week 3: 3,000 failures (30% budget consumed) -- warning
Total:  10,000 failures -- budget exhausted!

Consequence: freeze feature deployments, focus on reliability
New features increase risk (consume budget) -- managed tradeoff
```

Code Examples

Prometheus + Spring Boot Setup:

```
# pom.xml
<dependency>
    <groupId>io.micrometer</groupId>
    <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>

# application.yml
management:
  endpoints:
    web:
      exposure:
        include: health,prometheus,info,metrics
  metrics:
    distribution:
      percentiles-histogram:
        http.server.requests: true    # enable histogram for p95/p99
      percentiles:
        http.server.requests: 0.5,0.95,0.99

# Custom business metric
@Service
public class OrderService {
    private final Counter orderCounter;
    private final Timer orderTimer;

    public OrderService(MeterRegistry registry) {
        this.orderCounter = registry.counter("orders.placed.total",
            "status", "success");
        this.orderTimer = registry.timer("order.processing.duration");
    }

    public Order placeOrder(OrderRequest request) {
        return orderTimer.record(() -> processOrder(request));
    }
}
```

PromQL Queries for Java SLOs:

```
# Error rate (5xx / total requests over 5 min window)
rate(http_server_requests_seconds_count{status=~"5.."}[5m])
  /
rate(http_server_requests_seconds_count[5m])

# P99 latency (requires histogram)
histogram_quantile(0.99,
  rate(http_server_requests_seconds_bucket[5m]))

# JVM heap usage
jvm_memory_used_bytes{area="heap"}
  /
jvm_memory_max_bytes{area="heap"}

# GC pause rate
rate(jvm_gc_pause_seconds_sum[5m])

# Recording rule (precompute expensive query)
groups:
  - name: java_api
    interval: 1m
    rules:
      - record: job:http_errors:rate5m
        expr: rate(http_server_requests_seconds_count{status=~"5.."}[5m])
```

OpenTelemetry Java Auto-Instrumentation:

```
# Dockerfile: add OTel agent
FROM eclipse-temurin:21-jre
COPY --from=build /app/app.jar /app.jar
COPY opentelemetry-javaagent.jar /otel-agent.jar

ENV JAVA_TOOL_OPTIONS="-javaagent:/otel-agent.jar"
ENV OTEL_SERVICE_NAME="java-api"
ENV OTEL_EXPORTER_OTLP_ENDPOINT="http://otel-collector:4317"
ENV OTEL_TRACES_SAMPLER="parentbased_traceidratio"
ENV OTEL_TRACES_SAMPLER_ARG="0.1"   # sample 10% of traces

ENTRYPOINT ["java", "-jar", "/app.jar"]

# Auto-instrumented:
# - HTTP server (Spring MVC, WebFlux)
# - JDBC (database queries)
# - Kafka consumer/producer
# - gRPC
# - Spring Scheduled tasks
```

Interview Q&A – Observability & SRE Practices

Q1. What are the three pillars of observability?

Metrics: aggregated numerical measurements over time (request rate, latency p99, error rate, JVM heap). Efficient storage, good for alerting and dashboards. Logs: timestamped records of discrete events (HTTP requests, exceptions, audit trail). Verbose, searchable. Traces: request paths across Java microservices (spans showing each service/DB call). Essential for debugging distributed latency. Together: metrics for detecting issues, logs for diagnosing, traces for pinpointing.

Q2. How does Micrometer auto-expose Spring Boot metrics?

Adding micrometer-registry-prometheus dependency and exposing /actuator/prometheus: Micrometer auto-instruments Spring Boot (HTTP request counts/durations by URI/status), JVM (heap/GC/threads/classes), HikariCP (active/idle/pending connections), Spring Cache (hit/miss rates), Spring Batch (job durations). Prometheus scrapes every 15s. management.metrics.distribution.percentiles-histogram enables histogram for histogram_quantile() PromQL.

Q3. What is histogram_quantile in PromQL?

Prometheus histograms record observations in buckets (e.g. response times in 0.005s, 0.01s, 0.025s, 0.05s, 0.1s, 0.25s, 0.5s, 1s, 2.5s, 5s, 10s). histogram_quantile(0.99, rate(http_server_requests_seconds_bucket[5m])) computes approximate P99 from bucket counts. More accurate with more buckets. Alternative: percentiles micrometer setting exports pre-calculated percentiles as gauges (less accurate but simpler PromQL).

Q4. How does OpenTelemetry Java auto-instrumentation work?

OTel Java agent (-javaagent:otel-agent.jar) uses Java instrumentation API to bytecode-inject spans into Spring MVC, JDBC, Kafka, Redis, gRPC without code changes. On HTTP request: creates root span, adds HTTP attributes (method, URL, status). JDBC query: creates child span with SQL statement, DB type, table. Propagates trace context via W3C traceparent header to downstream Java services. Exports spans via OTLP to Jaeger/Tempo/Zipkin.

Q5. What is an SLO and how is it different from SLA?

SLI (indicator): measurable metric -- error rate, P99 latency, availability. SLO (objective): internal target for an SLI -- 99.9% requests under 200ms in 30 days. SLA (agreement): contractual commitment to customers with financial penalties for breach -- typically less stringent than internal SLO. Setting SLO more aggressive than SLA creates buffer. Error budget = 100% - SLO% = acceptable failure budget per period.

Q6. How does error budget guide reliability decisions?

Error budget consumed = actual failures / allowed failures. Fast-moving team (many deployments): consumes budget. When budget exhausted: freeze feature deployments, prioritise reliability work (reduce toil, add retries, improve monitoring). Budget remaining: team can take more deployment risk. Data-driven conversation: product vs engineering alignment on risk tolerance. Avoids both over-engineering (100% uptime impossible) and under-investing in reliability.

Q7. How do Alertmanager routes and inhibitions work?

Alertmanager receives Prometheus alerts and routes based on labels: route: receiver: pagerduty match: severity=critical; route: receiver: slack match: severity=warning. Grouping: batch similar alerts (all pods down -> one page). Inhibition: silence downstream alerts when root cause firing -- inhibit: source_match: alertname=DBDown; target_match: alertname=HighErrorRate (DB down causes high error rate; only alert on DB down, not symptom).

Q8. What Prometheus recording rules should you create for Java services?

Record expensive or frequently-evaluated queries: error rate per service (job:http_errors:rate5m), p99 latency per endpoint (job_uri:http_latency_p99:rate5m), JVM heap utilisation, DB connection pool saturation. Recording rules pre-compute at evaluation interval -- dashboards and alerts use fast pre-computed metric instead of re-evaluating expensive PromQL on every dashboard refresh.

Q9. How do you implement distributed tracing across Java microservices?

OTel Java agent instruments each service. Trace context (W3C traceparent header) propagated automatically on HTTP (RestTemplate, WebClient) and Kafka messages. Each service creates spans representing work units. Spans linked by trace ID. OTel Collector aggregates and exports to Jaeger. Grafana Tempo: integrated with Loki logs (trace-to-log linking). Spring Boot: trace ID auto-added to log MDC by Micrometer Tracing + OTel bridge.

Q10. What is tail sampling in OpenTelemetry and when should you use it?

Head sampling: decide at trace start whether to sample (e.g. 10% random). Always misses interesting traces. Tail sampling: buffer all spans, decide after trace completes based on outcome -- always sample errors and slow traces (>500ms), sample only 1% of

fast successful traces. OTel Collector tail_sampling processor implements this. Best for Java APIs where 99% of traces are fast and healthy -- store 100% of error and slow traces for debugging.